

Rapport de projet : Contrôle adaptatif B appliqué à un quadricoptère

Cours SYS802



Sommaire :

1.	Introduction.....	3
2.	Contrôle classique du quadricoptère.....	4
2.1.	Modélisation du quadricoptère.....	4
2.2.	Architecture de contrôle.....	6
2.3.	Modélisation Simulink et résultats.....	10
2.4.	Commande du quadricoptère réel.....	12
3.	Contrôle du drone avec la méthode adaptative.....	12
3.1.	Introduction à la méthode adaptative.....	12
3.2.	Modèle de référence.....	13
3.3.	Mécanisme d'apprentissage.....	14
4.	Commande B.....	16
4.1.	Commande B théorique.....	16
3.2	Commande B pour le quadricoptère.....	20
1.	Commande adaptative B.....	24
2.	Conclusion.....	26
3.	Bibliographie.....	27
4.	Annexes.....	28
4.1.	Sous modèle du code simulink.....	28
4.2.	SystemIdentification.m.....	30
4.3.	Annexe 2.....	31
4.4.	Annexe 3.....	32

1. Introduction

De nos jours, les commandes classiques sont largement utilisées dans le domaine du contrôle des systèmes. Nous pouvons cependant nous interroger : pourquoi ne pas plutôt opter pour une commande adaptative ? Les commandes classiques, telles que le correcteur PID, s'appuient sur des paramètres fixes pour compenser l'erreur en modifiant l'entrée du système. La mise en place de contrôleur quasi linéaire se repose sur le principe de l'utilisation d'un gain élevé pour atteindre la sortie souhaitée, notamment dans le cas des systèmes d'ordre 2. Cependant, l'application d'un gain trop grand peut générer de l'instabilité et des perturbations du système. En revanche, une commande adaptée, telle que la commande B, serait plus propice à ce genre de cas.

Dans le cadre du cours SYS802 de méthodologie avancée de commandes, donné par le professeur M. Bensoussan, nous avons étudié différentes méthodes de commande, dont la commande B qu'il a lui-même développée. Pour donner un aspect plus pratique à cet apprentissage, nous devons réaliser un projet appliquant la commande B à un système réel. Nous avons donc choisi d'associer la commande B et la commande MRAC pour commander un quadricoptère. Une fois les notions théoriques de la commande B assimilées en cours, nous avons commencé le projet en rédigeant un rapport préliminaire. Celui-ci visait à segmenter et clarifier les différentes parties, établir un échéancier, et fixer des objectifs pour le bon déroulement de notre projet. De plus, une salle au département des génies électrique a été mise à notre disposition. De même, les composants d'un drone de type F450 DJI nous ont été transmis pour l'assembler et le mettre en état de marche. Ce drone devait nous permettre de tester la commande B+MRAC développé sur un système réel et de prendre des mesures afin de valider l'utilité d'une commande B adaptative. Nous avons aussi dû commander des pièces pour construire un support destiné à retenir le quadricoptère. À mi-parcours, une présentation intermédiaire nous a permis de faire le point sur nos avancées et sur les étapes restantes. Enfin, la dernière étape, si nous avons le temps, était d'appliquer ces commandes sur le système réel, prendre des mesures et de comparer toutes ces données collectées. Ce rapport fait part du cheminement de notre projet et de nos résultats.

2. Contrôle classique du quadricoptère

L'une des premières étapes de notre projet fut d'implémenter une commande classique du drone. Pour rentrer au plus vite dans le sujet principal du projet, c'est-à-dire la commande du drone avec une commande B adaptative, nous avons décidé de choisir l'architecture de commande la plus simple qu'il soit. De nombreuses références et d'exemples sont disponibles en ligne. Nous nous sommes particulièrement aidés d'une série de cours et d'exemple fournie par Matlab dont le lien est [ici](#).

2.1. Modélisation du quadricoptère

Le quadricoptère est un engin volant commandé par quatre moteurs placés aux extrémités du châssis. Ces quatre moteurs sont coordonnés par un contrôleur pour que le quadricoptère puisse se déplacer dans l'espace. Le quadricoptère peut effectuer trois translations suivant x , y et z , et peut effectuer trois rotations nommées : pitch, roll et yaw. On dit alors qu'il a 6 degrés de liberté. Pour faciliter notre projet nous nous restreignons à un déplacement en « hovering » c'est-à-dire que nous considérons seulement sa position (x, y, z) et le yaw .

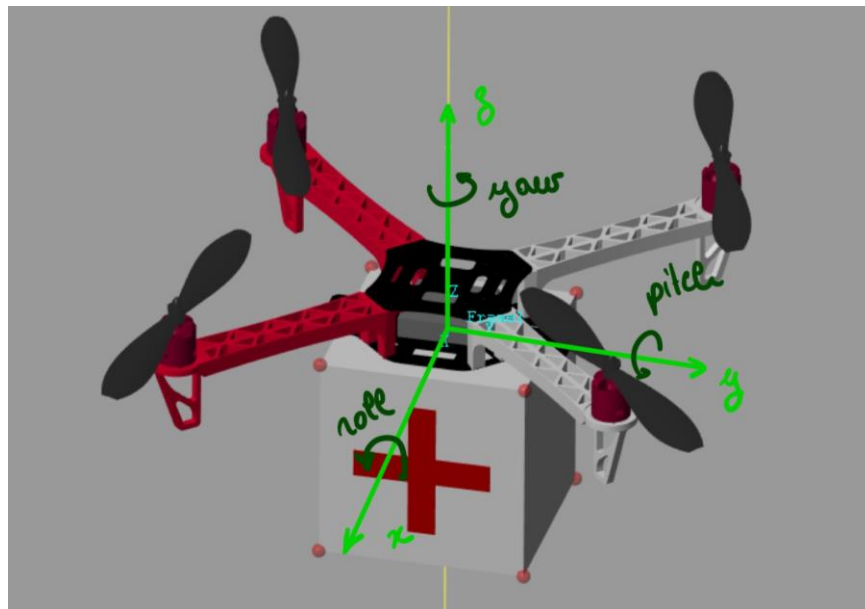


Figure 1 : Quadricoptère avec ces 6 degrés de libertés

Le quadricoptère est un système complexe à contrôler pour deux raisons. La première est que ce système est très instable et évolue souvent dans des environnements extérieurs sujet à de nombreuses perturbations (vent, pluie, obstacles, etc...). La deuxième est que les degrés de liberté en translation x et y sont dépendant des angles roll et pitch du système. En d'autres termes, pour pouvoir se déplacer le quadricoptère doit nécessairement s'incliner dans une certaine direction. On ne pourra donc jamais commander chaque degré de liberté indépendamment.

Après en avoir plus appris sur les quadricoptère, il nous a fallu implémenter le modèle du drone sur Matlab simulink. Par chance, nous sommes tombés sur un modèle simulink très précis du drone que nous avons dans le laboratoire.

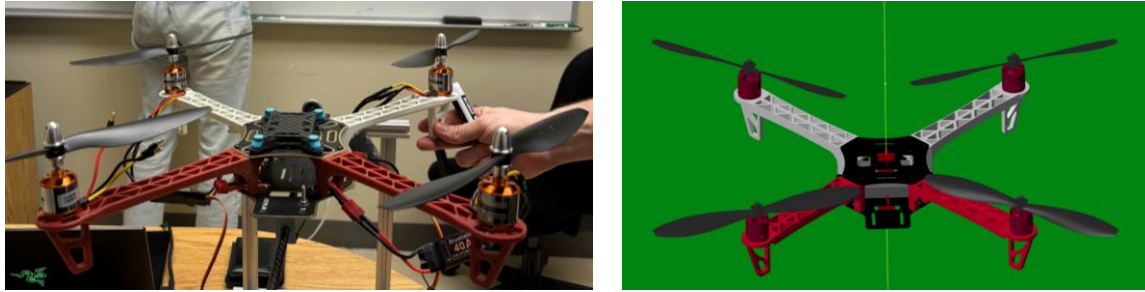


Figure 2 : Comparaison modèle simulink et quadricoptère réel

Ce modèle se présente sous forme d'un bloc simulink qui prend en entrée le bus cmd (pour commande) et prend en sorti le bus m (pour masse). Le bus commande comprends quatre vitesses angulaires pour chacun de moteur noté w_1 w_2 w_3 et w_4 . Le bus masse comprends la position x , y , z et yaw qui sont les seules caractéristiques du drone quand celui-ci se déplace en « hovering ».

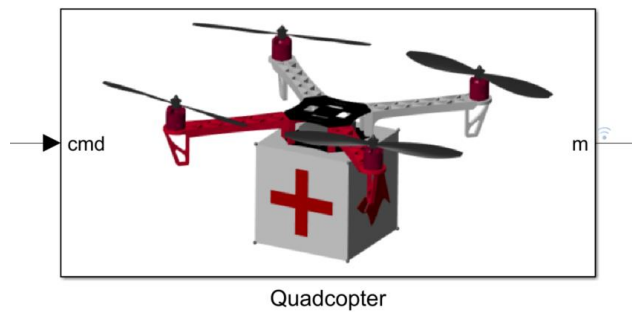


Figure 3 : Bloc simulink du quadrocoptère

Ce modèle est très détaillé et comprend le contrôleur et ses fonctionnement interne, les fonctionnements électromagnétiques des moteurs, la résistance des matériaux à l'effort et beaucoup d'autres phénomènes. De plus ce modèle inclut une simulation avec trois angles de vues dans un environnement sans perturbations. Le modèle est très détaillé mais donc aussi très complexe et aussi très non-linéaire ce qui va rendre plus difficile sa commande et qui par la suite va nous rendre la tâche plus difficile.

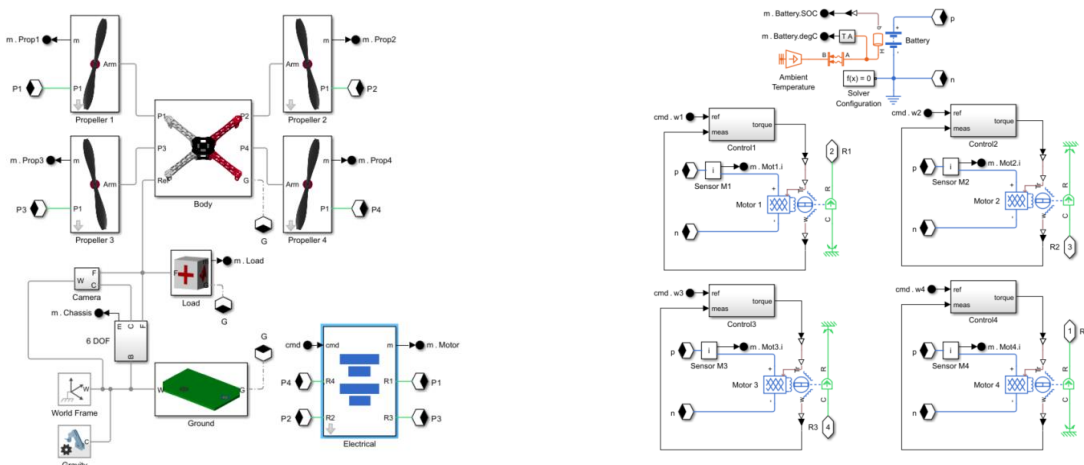


Figure 4 : Modèle général du quadricoptère et modèle détaillé du contrôleur

2.2. Architecture de contrôle

Il existe de nombreuses architectures de contrôle d'un drone, la plus immédiate et celle que nous avons choisi, se décompose en trois blocs : le *motor mixer*, l'*inner loop* et l'*outer loop*. Ces trois blocs sont des sous-systèmes avec des objectifs très précis, détaillé dans les parties suivantes. Les codes de chaque sous modèle sont d'ailleurs disponibles en annexe.

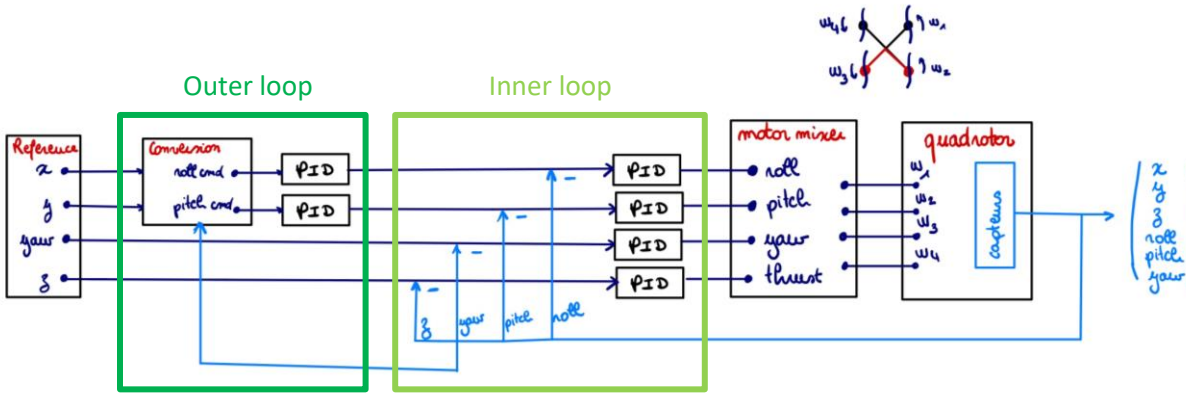


Figure 5 : architecture générale du contrôle classique du drone

2.2.1. Motor mixer

Le moteur mixeur est une partie du contrôleur qui permet la répartition des commandes : *thrust*, *roll*, *pitch* et *yaw* sur les vitesses angulaires w_1 , w_2 , w_3 et w_4 . Chaque commande est une combinaison de vitesse de rotation de quatre moteurs :

- *Thrust* : indique une élévation d'altitude, qui nécessite une certaines vitesse rotation pour tous les moteurs (1, 2, 3 et 4)
- *Roll* : une rotation autour de l'axe y, nécessite que les deux moteurs latéraux (w_1 et w_3) aient une vitesse de rotation plus grande que les deux moteurs latéraux opposés (w_2 et w_4)
- *Pitch* : une rotation autour de l'axe x, nécessite que les deux moteurs arrière (w_3 et w_4) aient une vitesse de rotation plus grande que les deux moteurs avant (w_1 et w_2)
- *Yaw* : une rotation autour de l'axe z, nécessite que deux moteurs opposés aient une rotation moindre que les deux autres moteurs. Pour que le quadricoptère en tourne pas sur lui-même il est nécessaire que les moteurs opposés tournent dans des sens contraires.

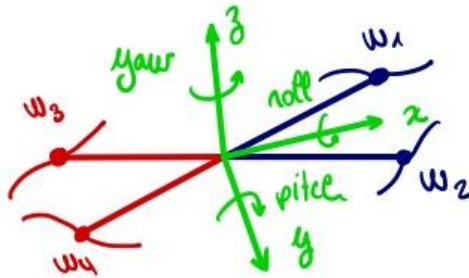


Figure 6 : Configuration des moteurs sur notre modèle de quadricoptère

On peut ainsi exprimer les vitesses de rotations en fonction des commandes avec les formules suivantes. Les valeurs des commandes : *thrust*, *roll*, *pitch* et *yaw* seront déterminées plus tard par les *l'inner-loop* et *l'outer loop*.

$$\begin{cases} w_1 = thrust + roll - pitch - yaw \\ w_2 = thrust - roll - pitch + yaw \\ w_3 = thrust + roll + pitch - yaw \\ w_4 = thrust - roll + pitch + yaw \end{cases}$$

2.2.2. Inner loop

L'*inner-loop* est en amont du motor mixer et permet d'assurer de commander le drone en *thrust*, en *roll*, en *pitch* et en *yaw*. A ce stade le drone maintient une position stable mais ne pas encore resté sur un point fixe car rien ne l'y contraint. L'*inner-loop* est une boucle de contrôle constitué de quatre PID qui doivent être réglé pour stabiliser le quadricoptère.

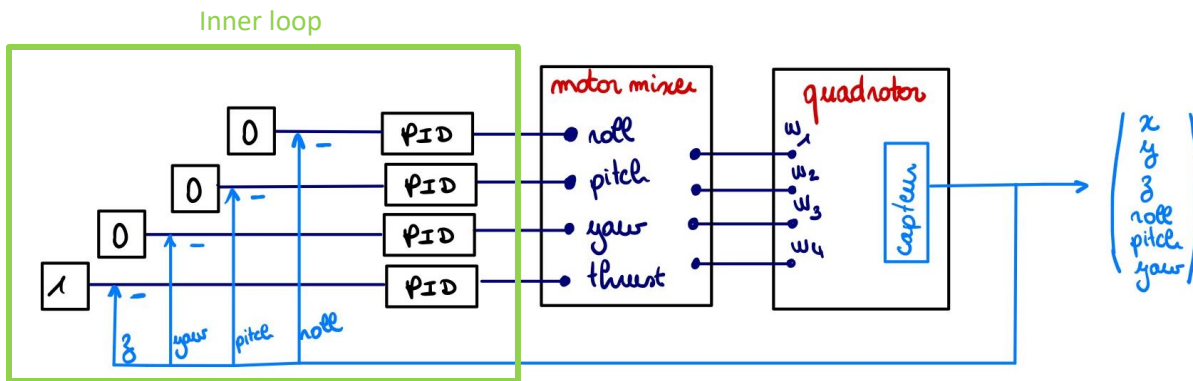


Figure 7 : Schéma bloc avec l'inner-loop et le motor mixer

Régler les PID est une opération fastidieuse et chronophage, et il n'est pas possible d'utiliser la fonction « autotune » de Simulink car notre modèle est trop instable. Il faut donc régler chaque paramètre et faire tourner la simulation manuellement jusqu'à obtenir une réponse à la consigne satisfaisante. Pour régler ces PID, nous avons utilisé comme consigne un échelon en altitude d'un mètre, comme indiqué sur le diagramme ci-dessus.

p_thrust = 2500;	p_yaw = 1000;	p_pitch = 1000;	p_roll = 1000;
i_thrust = 2000;	i_yaw = 300;	i_pitch = 100;	i_roll = 100;
d_thrust = 1000;	d_yaw = 100;	d_pitch = 500;	d_roll = 500;

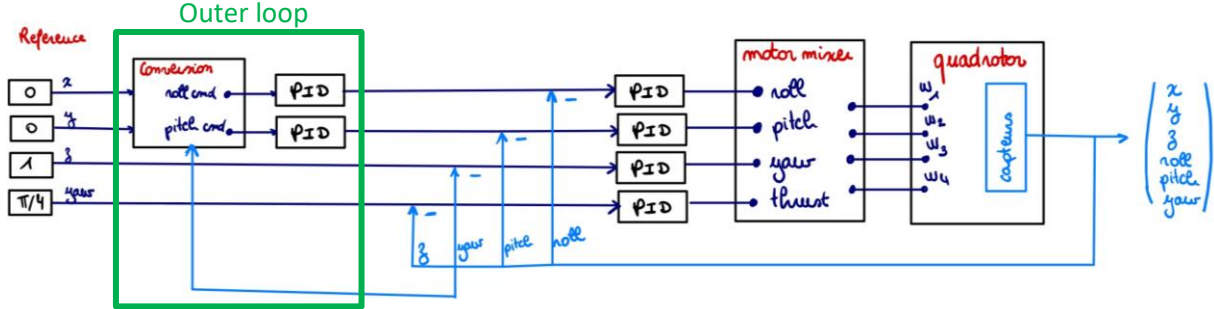
Figure 8 : Valeurs de PID trouvées de manière empirique

Pour l'*inner-loop*, il nous ait venu plusieurs fois à l'idée d'intégrer la commande B à la place des PID. Pour utiliser la commande, il nous fallait connaître la fonction de transfert globale du système. Cette fonction de transfert était détaillé dans l'article « A new fast compensator design applied to a quadricopter ». En reprenant cette fonction de transfert et en récupérant les mêmes coefficients que dans l'article avec quelques adaptations le système n'était pas stable. En effet, la fonction de transfert n'était pas adaptée à notre modèle de drone trop complexe et trop non-linéaire. Nous avons essayé d'autres fonctions extraites

de d'autres articles mais en vain. Nous sommes donc restés sur la méthode de contrôle classique comprenant des PID. Nous intégrerons la commande B plus tard en entrée du système pour réguler l'entrée et l'adaptation du processus. Nous y reviendrons dans une prochaine partie.

2.2.3. Outer loop

L'outer-loop est une boucle qui vient s'ajouter à l'inner-loop. Elle permet de stabiliser la position du quadricoptère par rapport à un point fixe. Elle est constituée d'un bloc de **conversion** et de deux PID.



Le bloc de **conversion** permet de convertir un vecteur depuis le repère du monde (internal frame) vers le repère du quadricoptère (body frame). Dans notre cas, on convertit le vecteur $\vec{P}$ indiquant un point sur lequel doit se rendre le quadricoptère du repère du monde défini par $\langle x_i, y_i \rangle$ au repère du quadricoptère défini par $\langle x_b, y_b \rangle$ avec l'opération suivante

$$\vec{P}_{\langle x_b, y_b \rangle} = \begin{pmatrix} \cos(yaw) & -\sin(yaw) \\ \sin(yaw) & \sin(yaw) \end{pmatrix} \vec{P}_{\langle x_i, y_i \rangle}$$

Cette opération n'est en fait que la rotation du vecteur d'un angle yaw . Cela permet d'exprimer le vecteur $\vec{P}$ dans la base $\langle x_b, y_b \rangle$ qui en lien direct avec le *roll* et le *pitch*.

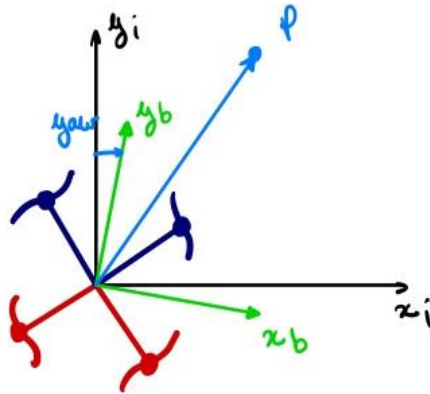


Figure 9 : Conversion vecteur $\vec{P}$ d'une base à une autre

De plus, il faut régler les deux PID de manière empirique ce qui est encore laborieux et chronophage à cause de la complexité du système. Pour régler les PID on prend une entrée fixe $x = 0, y = 0, z = 1$ et $yaw = \pi/4$. On obtient comme valeur de PID les valeurs suivantes :


```

p_control = 0.1;
i_control = 0.0;
d_control = 5;

```

```

p_control = 0.1;
i_control = 0.0;
d_control = 5;

```

Figure 10 : Valeurs PIDs pour le maintien de position

2.2.4. Entrée du système

Maintenant que nous avons toute l'architecture de contrôle, on peut s'intéresser à l'entrée du système. La solution la plus simple pour faire se déplacer le drone et de lui donner en entrée des points de déplacement (en rouge) qu'il doit rejoindre. Chaque entrée est alors considérée comme un step d'une certaine valeur que l'on peut décaler dans le temps en fonction du moment pour lequel on veut que le quadricoptère rejoigne cette position.

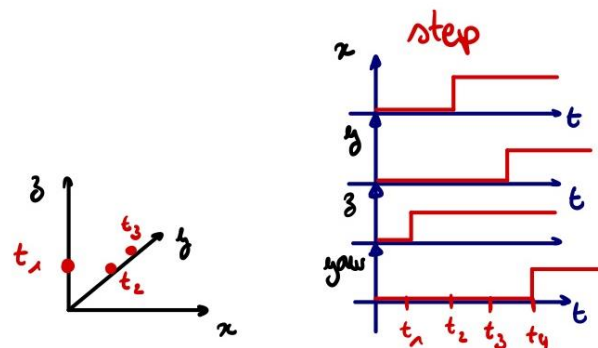


Figure 11 : Exemple de déplacement du quadricoptère avec une entrée échelon

Nous avons aussi pensé à une autre méthode pour commander notre quadricoptère, une méthode qui donnerait de meilleurs résultats. Cette méthode consiste à mettre des points intermédiaires (en vert) entre chaque point de déplacement (en rouge) pour avoir un déplacement plus stable et plus précis. On positionne les points de sorte que déplacement dans le temps ait la forme d'une fonction cubique. La fonction cubique est une fonction qui permet d'avoir une accélération et une décélération constante, ce qui rend le quadricoptère plus stable qu'une accélération brutale.

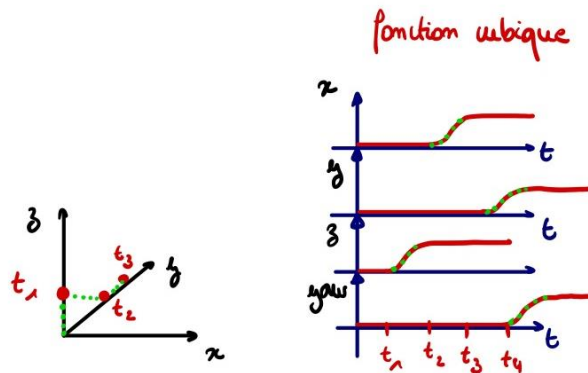


Figure 12 : Exemple de déplacement du quadricoptère avec une entrée cubique

Cette fonction cubique peut être définie, si on considère seulement un déplacement selon x de la façon suivante :

$$x(t) = x_s + s(t)(x_f - x_s)$$

Avec x_s la valeur de x initiale, x_f la valeur de x finale, celle que l'on veut rejoindre et $s(t)$ une fonction cubique variant entre 0 et 1 et dont l'expression est la suivante :

$$s(t) = \frac{3t^2}{T^2} - \frac{2t^3}{T^3}$$

Avec t le temps courant et T le temps que doit mettre le quadricoptère à atteindre la cible. Cette expression si elle est dérivée une fois donne une fonction cubique et si elle est dérivée deux fois nous donne une fonction linéaire, qui correspond à l'accélération que l'on veut donner au quadricoptère pour assurer sa stabilité.

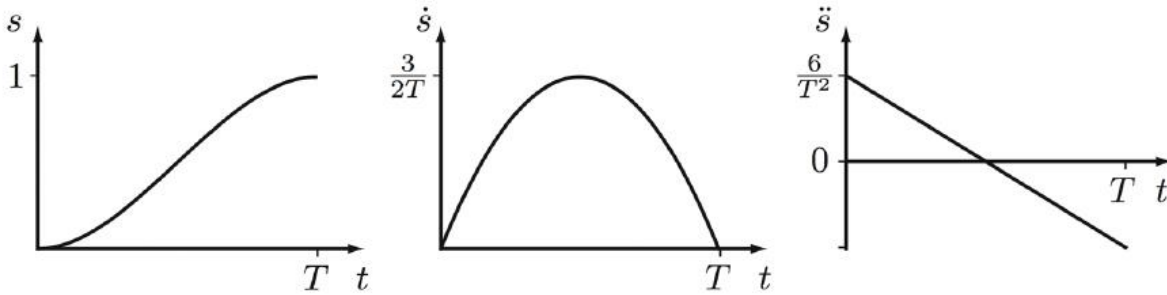


Figure 13 : Dérivée temporelle de la fonction $s(t)$

2.3. Modélisation Simulink et résultats

Maintenant que nous avons vu toute l'architecture de contrôle du drone théorique, il est temps de l'implémenter un logiciel de simulation. Dans un premier temps, nous avons modélisé le contrôle sur simulink, qui semblait le plus simple car notre quadricoptère et son environnement y étaient modélisés. En plus sur simulink il est possible de directement convertir un modèle en code C/C++ avec la fonction `simulink coder`. Cette opération est nécessaire car notre contrôleur de vol ne comprend que ce langage.

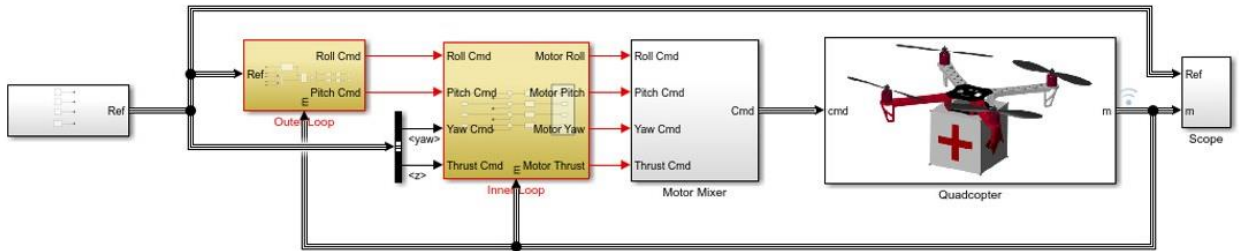


Figure 14 : Modèle simulink du contrôle du quadricoptère

En sortie de ce modèle simulink, nous avons d'une part la modélisation du drone dans son environnement et d'autre part les graphes de sorti indiquant la position du quadricoptère. Pour vérifier le fonctionnement de notre contrôle, nous appliquons au quadricoptère une consigne fixe $x = 0, y = 0, z = 1$ et $yaw = \pi/4$ ce qui nous donne les graphes suivants sur une fenêtre de 10s.

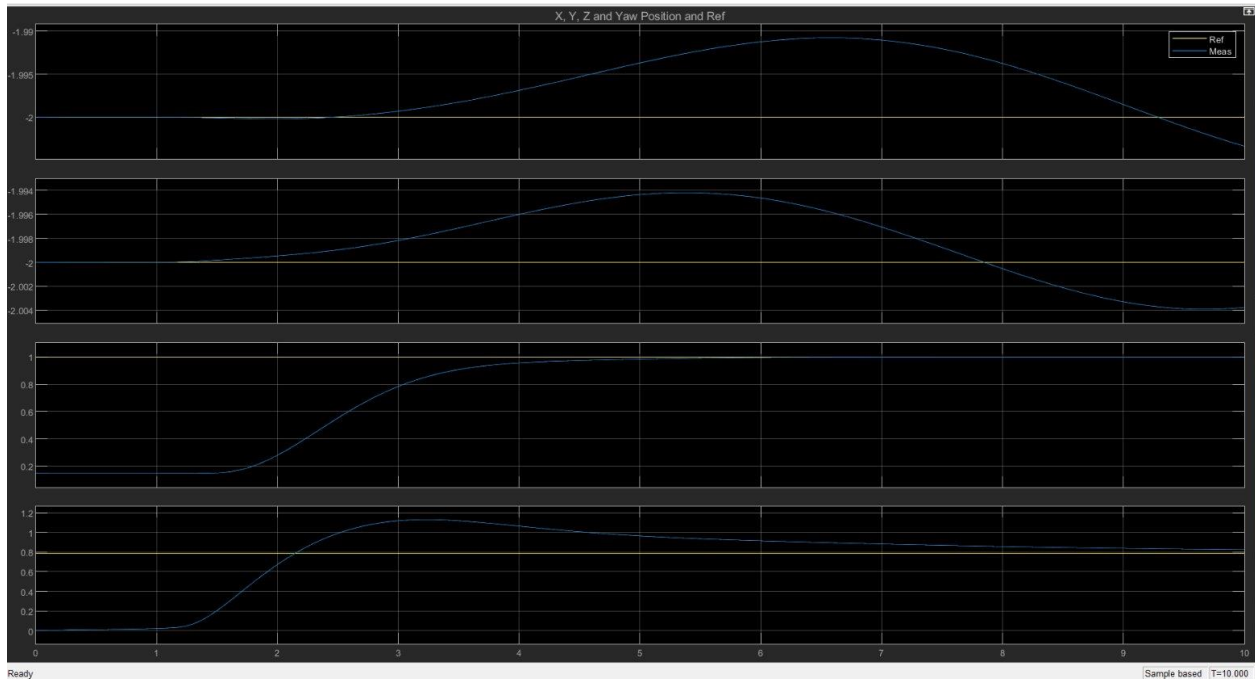


Figure 15 : Réponse du quadricoptère à des échelon fixes de $x = 0, y = 0, z = 1$ et $yaw = \frac{\pi}{4}$

La simulation du drone est disponible à ce [lien](#). Dans la simulation, on remarque que le drone à un paquet en dessous, la masse de ce paquet est considérée comme nulle pour les prochains tests.

En plus de Simulink, nous avons utilisé ROS. En effet, certaines parties du code sont difficile à mettre sur Simulink (notamment le suivi de trajectoire), donc nous avons aussi eu recours à ROS et au package `rotors_simulation` car coder directement en C/C++ peut faciliter certaines opérations.

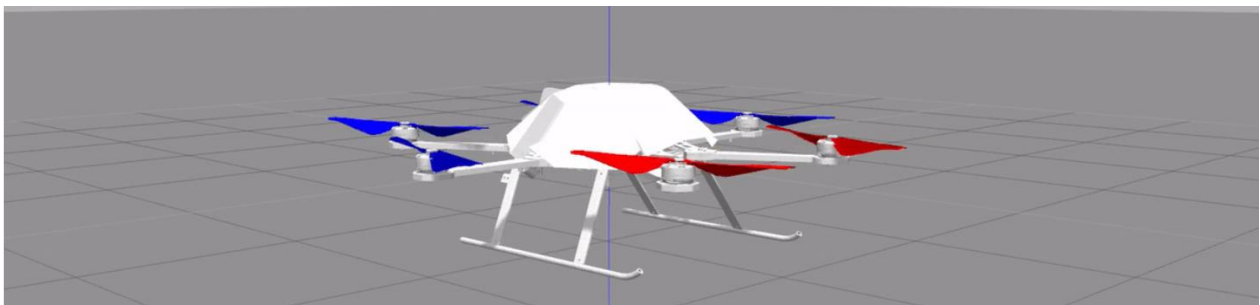


Figure 16 : quadricoptère du package "rotors" dans l'environnement Gazebo

Le projet d'utiliser ROS pour piloter notre drone fut rapidement abandonné car cela complexifiait la suite de notre projet. Néanmoins, nous avons réussi à piloter ce drone avec une trajectoire cubique décrite précédemment.

Le fait d'implémenter une commande d'un drone sur simulink est ce qui nous a pris le plus de temps. Souvent des concepts, comme la commande d'un quadricoptère, sont seulement expliqués théoriquement. Il faut pouvoir implémenter ces commandes nous-même ce qui prend du temps.

2.4. Commande du quadricoptère réel

Après avoir fait fonctionner le modèle du quadricoptère sur simulink, il nous a fallu penser à un moyen pour le mettre sur le quadricoptère réel. En effet, il y a une différence entre le quadricoptère réel et le modèle. Avec le modèle simulink, la position exacte du quadricoptère est connue alors qu'avec le quadricoptère réel il faut estimer sa position.

Pour estimer la position réelle du drone, on peut utiliser le GPS et un capteur IMU (Intertial Measurment Unit) déjà inclus dans le contrôleur de vol et combiné les deux mesures dans un filtre de Kalman. Nous avons eu le temps de réfléchir au filtre de Kalman mais nous n'avons pas pu le développer. De plus, il nous manquait du matériel et du temps pour implémenter la commande sur le quadricoptère réel.

3. Contrôle du drone avec la méthode adaptative

3.1. Introduction à la méthode adaptative

Maintenant que nous sommes intéressés à la commande classique du drone, intéressons-nous à la méthode adaptative MRAC. La méthode adaptative MRAC (Model Reference Adaptive Control) est une méthode populaire dans l'industrie qui permet d'adaptation de la commande en fonction des perturbations que subit le système. C'est presque une méthode de contrôle magique tant elle est facilement applicable à beaucoup de système. Pour s'adapter, cette commande utilise un modèle référence, qui est le modèle idéal qu'on aimerait que le système réel suive. Si le modèle réel s'éloigne du modèle de référence le système tend à le ramener vers le système réel en modifiant l'entrée du système réel. Cette modification de l'entrée du système réel est permise par un mécanisme d'apprentissage situé en bas schéma bloc ci-dessous. Dans notre cas par exemple, la perturbation pourrait être le dysfonctionnement de l'un des moteurs ou du vent ou de la pluie. Auquel cas le contrôle adaptatif ramènerai le drone sur sa trajectoire.

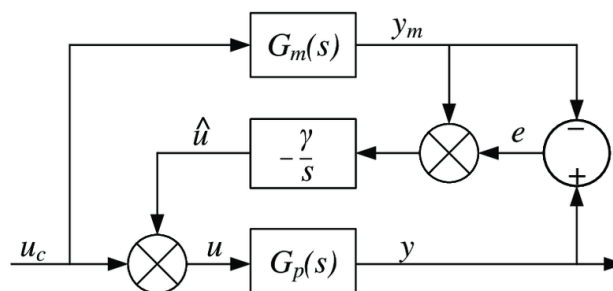


Figure 17 : Commande MRAC basée sur la règle MIT

3.2. Modèle de référence

Pour choisir notre modèle de référence (celui que doit suivre le système réel), on utilise les réponses du quadricoptères précédentes détaillé dans la partie **1.3 Modélisation Simulink et Résultat**. En effet dans cette partie le drone à un fonctionnement idéale, sans perturbations. On veut donc que notre système réel se comporte comme ce système idéal.

Maintenant il ne nous reste plus qu'à identifier le système du quadricoptère. Comme notre quadricoptère est très complexe nous ne pouvons pas déterminer une fonction de transfert directement. Il faut alors que nous identifions un système grâce à ces réponses indicielles. Nous choisirons de nous intéresser seulement aux réponses à un échelon en *yaw* et en *z*. Voici les réponses du quadricoptère extraites de simulink quand le quadricoptère est soumis à un échelon de $1rad$ sur le *yaw* et à un échelon de $1m$ sur *z*.

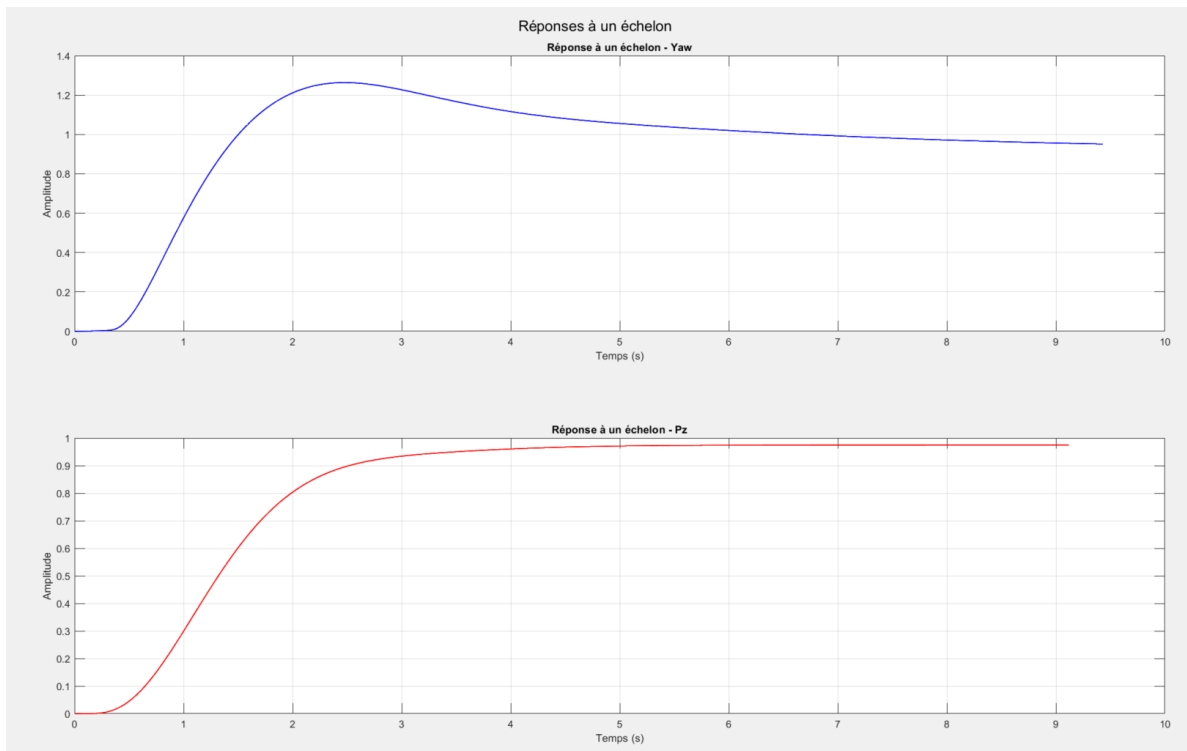


Figure 18 : Réponses indicielles filtrés du quadricoptère

Pour identifier le système à partir de ces réponses, on choisit un système d'ordre 2 avec deux pôles et on utilise le code `SystemIdentification.m` (disponible en annexe) avec la bibliothèque « System Identification Toolbox ». Pour que le signal soit exploitable, il faut le filtrer et enlever le début de la réponse car elle n'est pas linéaire. Voilà les deux fonctions de transfert que nous donne le code :

Fonction de transfert obtenue pour *yaw*

$$T_{yaw}(s) = \frac{3.915}{s^2 + 1.821s + 3.885}$$

Avec une estimation bonne à 89.19%

Fonction de transfert obtenue pour z

$$T_z(s) = \frac{0.864}{s^2 + 1.378s + 0.899}$$

Avec une estimation est bonne à 90.74%

Les estimations sont satisfaisantes mais ne sont pas très précises. Avec un meilleur filtrage des courbes initiales de meilleurs résultats pourraient être obtenus. Pour vérifier la validité de ces deux systèmes, nous avons simulé leurs réponses indicielles. Les résultats sont satisfaisants, on utilisera donc ces fonctions de transfert comme système de référence.

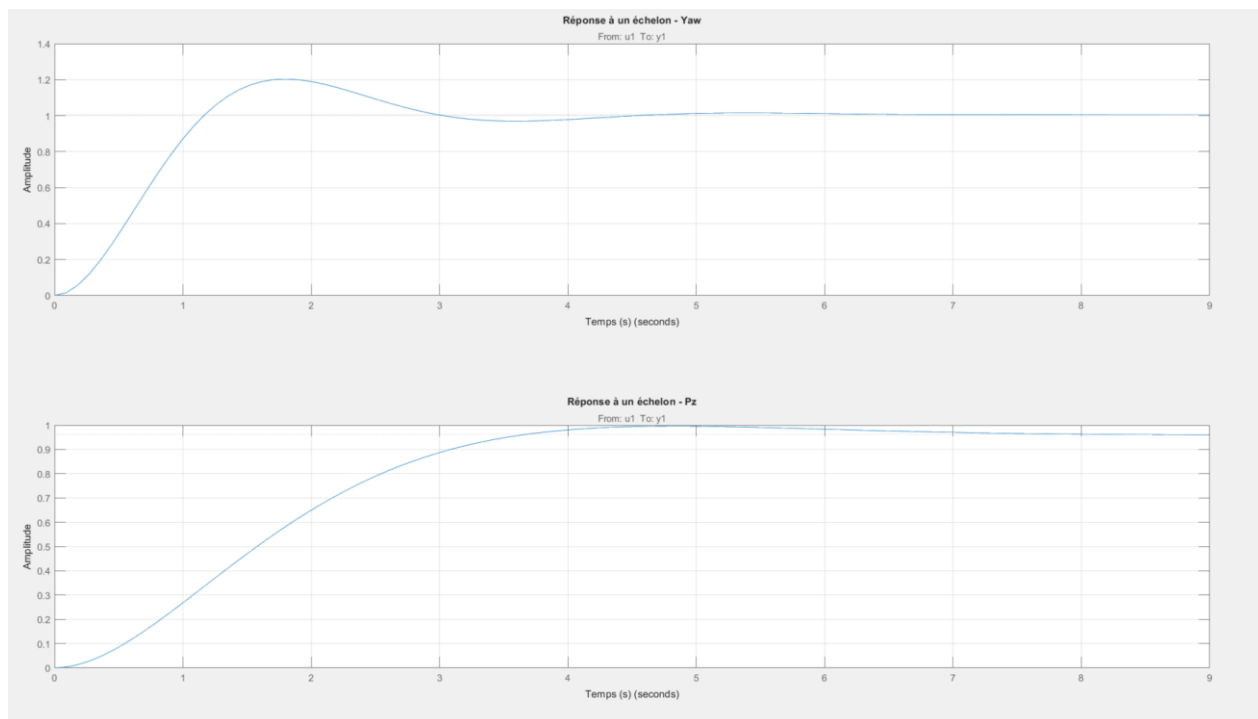


Figure 19 : Réponse indicielles des fonctions de transfert z et yaw

3.3. Mécanisme d'apprentissage

Pour que notre modèle puisse s'adapter, il faut implémenter une règle d'adaptation. Nous avons choisi la règle MIT qui est simple à implémenter et qui montre de bons résultats. La loi MIT repose un critère de minimisation, qui se veut de réduire l'erreur entre la sortie du système réel et la sortie du modèle de référence. Nous ne détaillerons pas son fonctionnement dans ce rapport. Pour être plus stable, il est aussi possible de rajouter un filtre passe bas pour obtenir une commande L1. Voici le schéma bloc global que l'on obtient, les détails de chaque sous système est disponible en annexe :

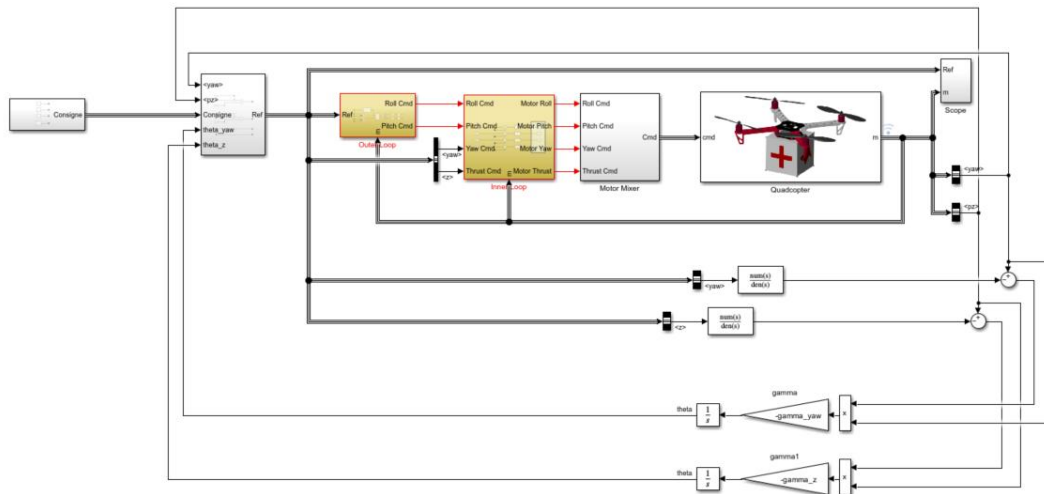


Figure 20 : Commande du quadricoptère avec la commande MRAC

Dans un premier essai, on règle l'apprentissage avec un gain de $\gamma = 0.5$, après en avoir essayé d'autres ce gain semble être le meilleur. On teste alors les performances de notre adaptation en soumettant notre quadricoptère à un créneau de $0.5m$ à une altitude de $1m$ et voilà le suivi qu'on obtient. La courbe jaune représentant le signal en entrée et la courbe bleu la réponse du drone.

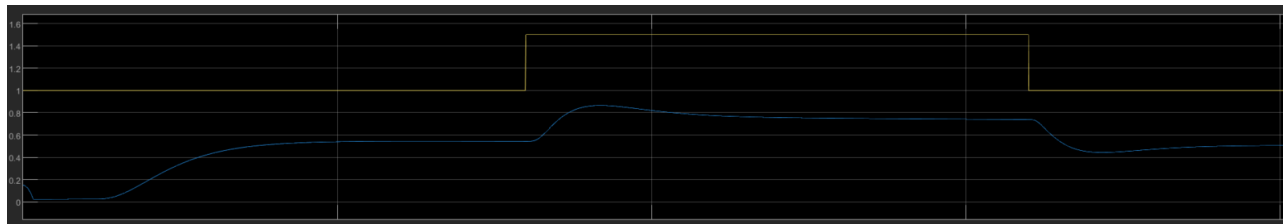


Figure 21 : Réponse en z du quadricoptère sans perturbation à un signal créneau

Ce suivi est bon avec un décalage vers le bas. Ce décalage peut être expliqué car la référence des altitudes n'est plus la même de $0.4m$. Maintenant pour tester l'adaptation du système on provoque une perturbation sur le système dans notre cas on alourdit le drone en mettant une masse en dessous de lui.



Figure 22 : Quadricoptère sans perturbation (à gauche) et avec perturbation (à droite)

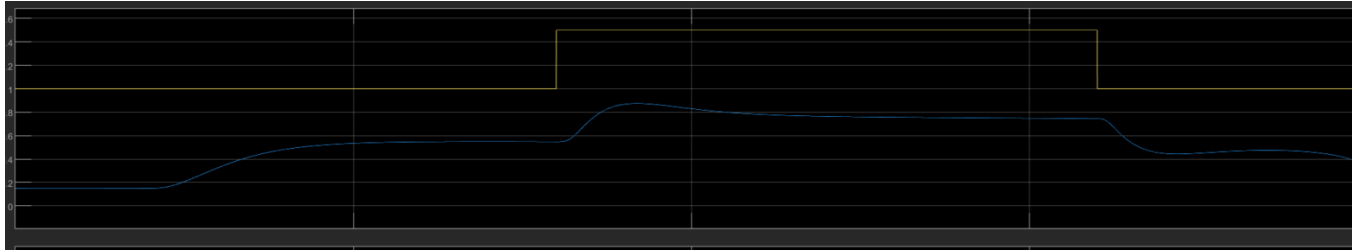


Figure 23: Réponse en z du quadricoptère avec perturbation à un signal créneau

On remarque alors que le drone suit quasiment la même élévation que précédemment, notre commande adaptative est donc fonctionnelle. En plus du test présenté, nous avons effectué une série de test en modifiant les caractéristiques d'un moteur, en alourdissant un bras du quadricoptère, etc... Tous les tests montrent bien l'efficacité de la commande adaptative MRAC à faire évoluer notre drone dans un environnement avec beaucoup de perturbations. Une propriété très utile si on veut faire voler le quadricoptère en extérieur.

4. Commande B

4.1. Commande B théorique

La commande B, proposée par David Bensoussan en 1984, est une méthode de commande qui permet d'obtenir simultanément d'excellentes performances et une grande robustesse. Contrairement aux approches classiques de commande, la commande B permet de combiner naturellement tous les critères de performance tels que le temps de réponse, le dépassement, le temps de stabilité, ainsi que les performances fréquentielles, incluant les marges de phase et de gain. Elle offre ainsi une solution qui répond à des besoins dans le domaine du contrôle des systèmes dynamiques.

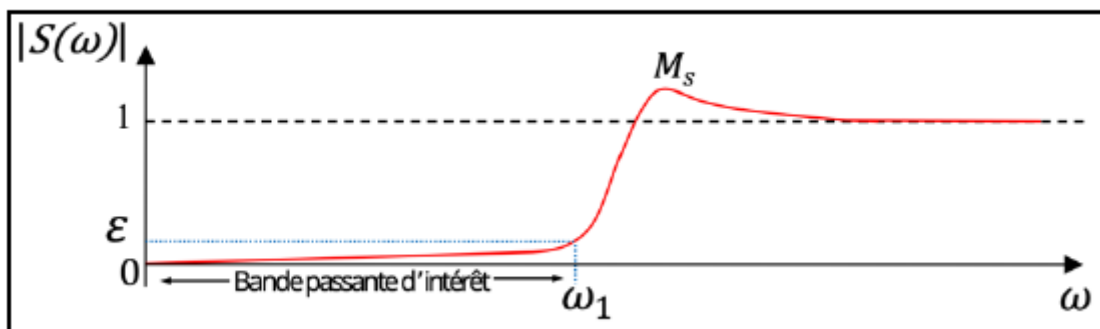


Figure 24 : Courbe fréquentielle de la sensibilité

Le principe de la commande B repose sur la recherche d'une fonction de sensibilité ayant une forme spécifique dans le domaine fréquentiel (voir figure 23). Plus précisément, la sensibilité doit être proche de zéro pour les basses fréquences, c'est-à-dire dans la bande passante définie entre $[0, \omega_1]$. Cela signifie que dans cette plage de fréquences, les perturbations et incertitudes ont un impact minimal sur le système, garantissant ainsi une excellente stabilité et précision. En revanche, pour les hautes fréquences, situées dans la bande $[\omega_1, \infty]$, la sensibilité doit atteindre une valeur de 1. Cette transition entre les deux bandes fréquentielles permet de maintenir la robustesse du système tout en évitant un gain excessif dans les hautes fréquences, avec la mise en place d'un gain maximum M_s , sinon cela pourrait amplifier le bruit ou créer des perturbations non désirées.

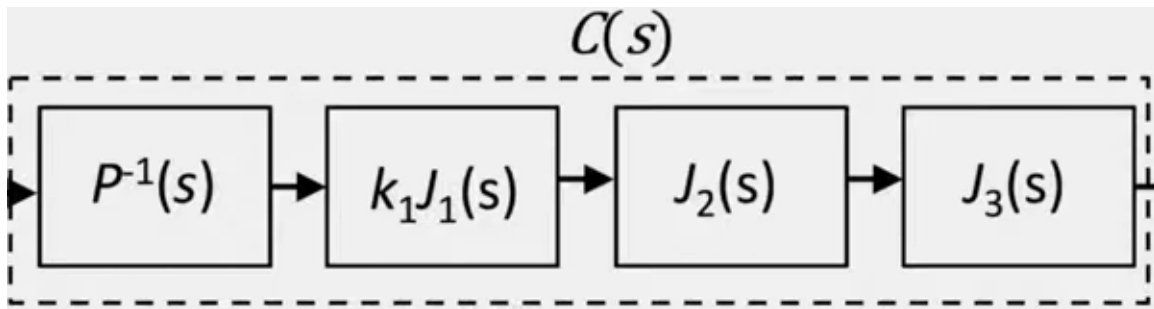


Figure 25 : Schéma bloc du compensateur B

Le compensateur B est composé de différentes fonctions de transfert qui ont chacune un rôle spécifique (voir figure 24). Ce compensateur est défini par l'équation suivante :

$$C(s) = J(s) * P^{-1}(s)$$

Dans cette équation, on retrouve le $P^{-1}(s)$ qui représente l'inverse de la dynamique du processus du système afin qu'il ne perturbe pas et ne crée pas du bruit pour le compensateur B. Par ailleurs, le $J(s)$ est utilisé pour façonner l'allure de la courbe de sensibilité S recherché. La représentation de $J(s)$ est donné par l'équation suivante :

$$J(s) = k_1 J_1(s) * J_2(s) * J_3(s)$$

Avec :

$$k_1 J_1(s) = \frac{k_1 * \omega_1}{s + \omega_1}$$

$$J_2(s) = \frac{s + z}{s + p}$$

$$J_3(s) = \left(\frac{\omega_2}{s + \omega_2} \right)^k$$

Le $k_1 J_1(s)$ permet d'imposer un haut gain à basse fréquence $[0, \omega_1]$ afin de minimiser la sensibilité S en dessous d'une valeur minimum ε . La fonction de transfert $J_3(s)$ a pour rôle de limiter le maximum M_s de la sensibilité afin de limiter l'apparition d'instabilité et de fixer l'ordre du compensateur pour vérifier qu'il soit réel et applicable lors de son implantation dans le système. Enfin, le $J_2(s)$ va permettre de réguler les fréquences intermédiaires de la courbe de sensibilité dans le but d'optimiser le temps de réponse temporel t_r du système.

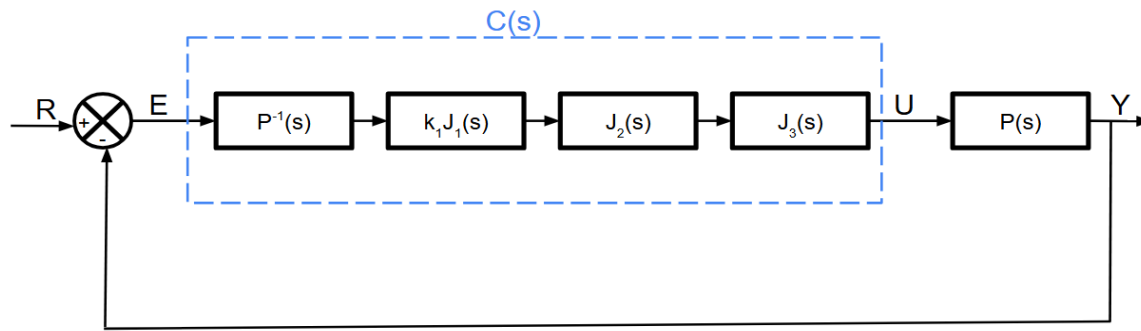


Figure 26 : Schéma bloc du système avec le compensateur $C(s)$ et le processus $P(s)$

Le compensateur B, représenté par le bloc $C(s)$, est intégré dans le système au niveau de la boucle de retour d'état, placé avant le bloc symbolisant le processus $P(s)$ (voir figure 25). Cette configuration permet de comparer la sortie réelle du système avec la sortie souhaitée en fonction de l'entrée donnée. En agissant ainsi, le compensateur joue un rôle crucial en ajustant dynamiquement l'entrée du système de manière à minimiser l'erreur entre la sortie obtenue et la sortie cible. Ce mécanisme de rétroaction garantit une meilleure précision du système en corrigeant les écarts en temps réel.

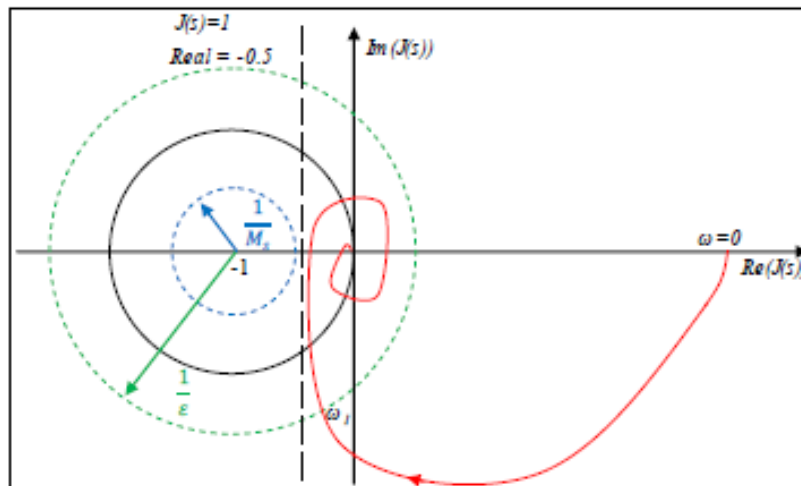


Figure 27 : Nyquist de $J(\omega)$ avec minimum ε et maximum M_s

En utilisant le logiciel Geogebra, nous représentons le diagramme de Nyquist de la boucle ouverte J (voir figure 26). Ainsi, le $J(s)$ est représenté par la courbe rouge selon l'axe des réels en abscisse et l'axe des imaginaires en ordonnée. Nous observons que la boucle ouverte conserve un haut gain à l'aide du compensateur jusqu'à la fréquence ω_1 qui correspond à l'intersection avec le cercle en pointillé vert de rayon $\frac{1}{\varepsilon}$. Dans ce cas, c'est la fonction de transfert $k_1 J_1(s)$ qui va agir pour réguler le $J(s)$. Ainsi, ce cercle symbolise la condition suivante :

Pour $\omega \leq \omega_1$

$$|[I + J(\omega)]^{-1}|_{\omega_1} < \varepsilon$$

$$|I + J(\omega)|_{\omega_1} < \frac{1}{\varepsilon} \quad \text{avec } \frac{1}{\varepsilon} > 1$$

Ensuite, notre boucle ouverte va suivre la droite verticale d'abscisse $\text{Re}(J(s))$ à -0.5, qui correspond à une transmission $T(s) \approx 1$. La transmission pouvant s'exprimer selon l'entrée R et la sortie Y à l'aide de l'expression suivante :

$$Y = T * R$$

Cela revient à dire que pour une transmission $T(s) \approx 1$, notre valeur de sortie va être équivalente à la valeur que l'on souhaite en entrée. De plus, cela permet à la courbe rouge de longer le cercle en pointillé bleu de rayon $\frac{1}{M_s}$ et qui a pour centre le point critique de coordonnées (-1, 0). C'est la fonction de transfert $J_3(s)$ qui va venir optimiser la valeur de notre maximum M_s . De plus, notre boucle ouverte J va devoir répondre au critère suivant :

Pour $\omega \geq \omega_1$

$$|[I + J(\omega)]^{-1}|_{\infty} < M_s$$

$$|I + J(\omega)|_{\infty} < \frac{1}{M_s} \quad \text{avec } \frac{1}{M_s} > 1$$

Cette condition représente la mise en place du maximum M_s sur la courbe de sensibilité afin de limiter l'instabilité du système à haute fréquence.

3.2 Commande B pour le quadricoptère

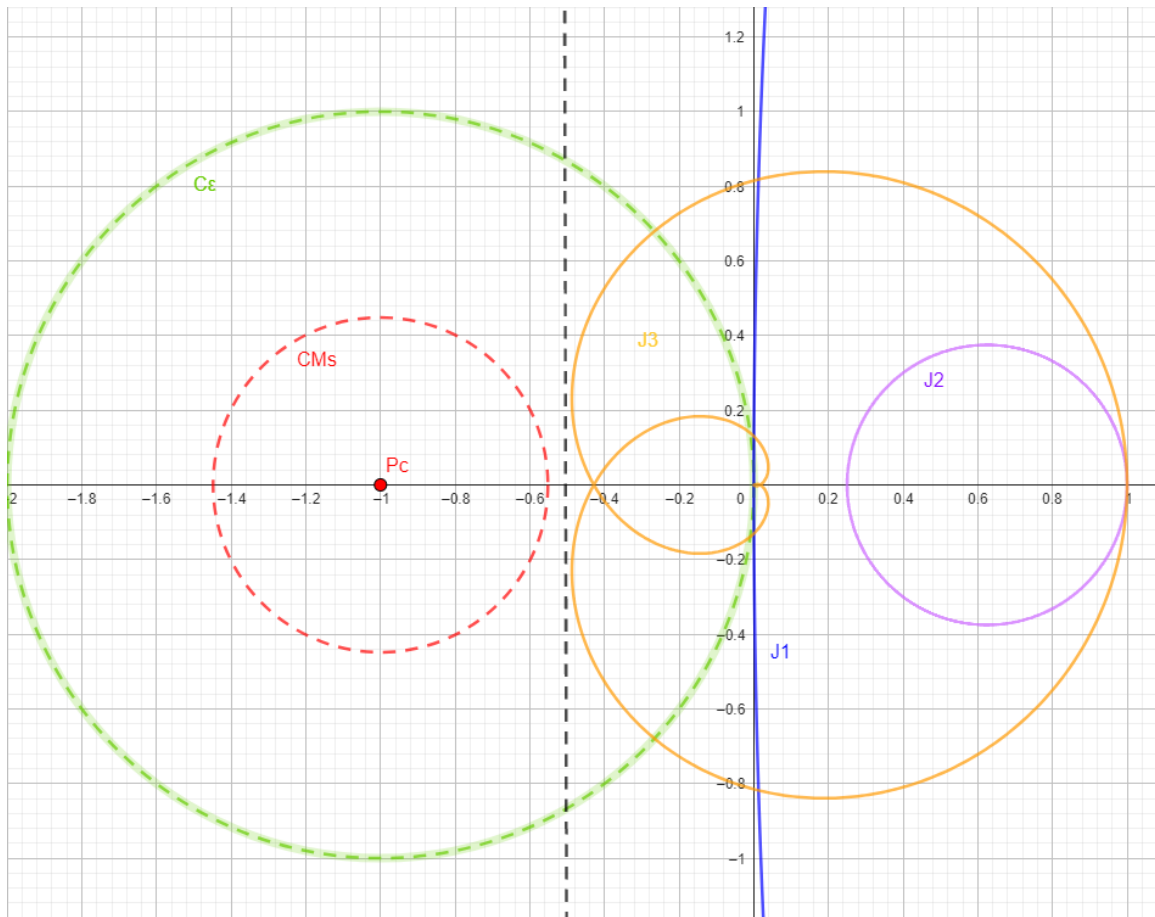


Figure 28 : Nyquist de J_1 , J_2 et J_3 sur Geogebra avec maximum M_s et minimum ε pour axe x et y

Dans le cadre de notre projet, nous devons trouver la commande B qui serait adapter au drone utilisé qui est le F450DJI. Ainsi, nous avons commencé par chercher les paramètres de $J_1(s)$, $J_2(s)$ et $J_3(s)$ selon les axes x, y, z et lacets. De ce fait, à l'aide du logiciel Geogebra, nous avons cherché à optimiser les valeurs de ω_1 , ω_2 , k_1 , k_2 , p et z pour obtenir le meilleur résultat possible pour la stabilisation du drone (voir figure 27). En modifiant les paramètres au travers de curseur et en respectant les critères décrits précédemment, nous avons obtenus les paramètres suivants :

Pour l'axe x et y :

$$\omega_1 = 0.07 \text{ rad/s}$$

$$\omega_2 = 34 \text{ rad/s}$$

$$k_i = 4$$

$$k_{li} = 53$$

$$z = 0.02$$

$$p = 0.08$$

Pour l'axe z

$$\omega_1 = 0.1 \text{ rad/s}$$

$$\omega_2 = 18 \text{ rad/s}$$

$$ki = 2$$

$$kli = 38$$

$$z = 0.1$$

$$p = 0.015$$

Pour le lacet

$$\omega_1 = 0.9 \text{ rad/s}$$

$$\omega_2 = 77 \text{ rad/s}$$

$$ki = 2$$

$$kli = 18$$

$$z = 0.1$$

$$p = 0.01$$

A partir de ces paramètres trouvés, nous pouvons réaliser un script sur le logiciel Matlab qui représente les fonctions de transfert $J(s)$ selon les différents axes du drone étudié (voir annexe 2). Pour la modélisation de nos commandes B, nous réalisons un schéma bloc sur Simulink de chaque compensateur. Tout d'abord, nous écrivons des lignes sur le script Matlab permettant de récupérer les numérateurs et dénominateurs de chaque $J_1(s)$, $J_2(s)$ et $J_3(s)$ pour reconstituer le $J(s)$ des axes que nous voulons corriger (voir annexe 3). Ensuite, nous créons un fichier Simulink sur lequel on crée des schéma bloc avec une entrée échelon, le compensateur B de chaque axe, une boucle de retour d'état et un bloc de prise de mesure en sortie du système (voir figure 28).

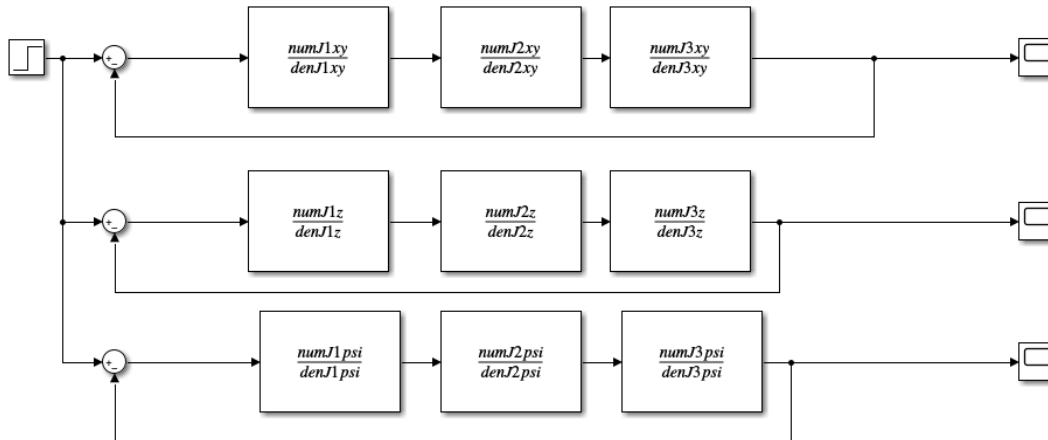


Figure 29 : Schéma bloc des compensateur B sur Simulink

Sur ce fichier Simulink, chaque ligne de blocs du schéma représente le compensateur B d'un axe du drone que nous souhaitons corriger. Puis, nous testons ces différents systèmes avec une même entrée échelon en testant la sortie (voir figures 29, 30 et 31).

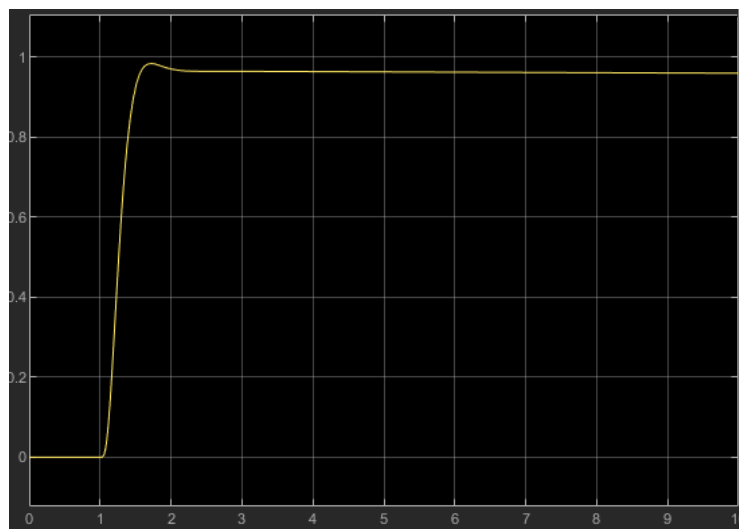


Figure 30 : Sortie de l'entrée échelon avec compensateur B pour l'axe x et y

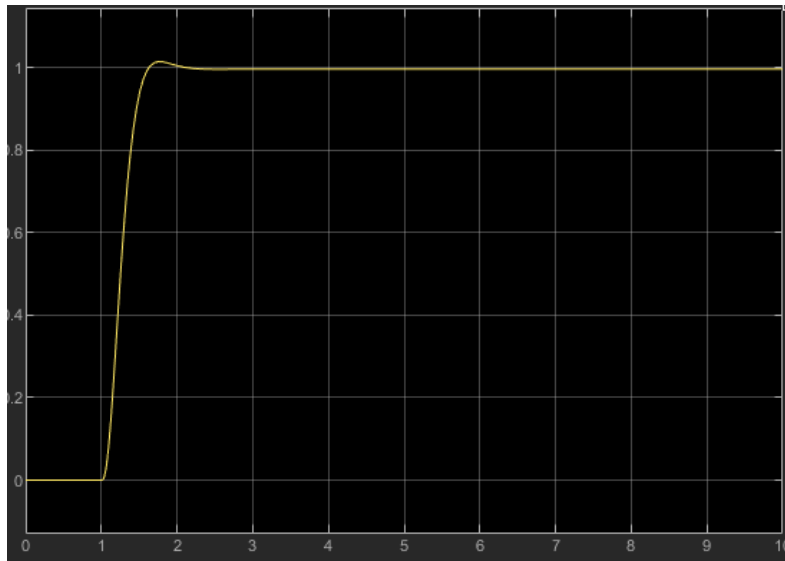


Figure 31 : Sortie de l'entrée échelon avec compensateur B pour l'axe z



Figure 32 : Sortie de l'entrée échelon avec compensateur B pour l'axe des lacets

En sortie des systèmes, utilisant les différents compensateur B, nous observons une réponse dont l'allure suit l'échelon unitaire appliqué en entrée. De plus, nous observons sur les simulations que le temps de stabilisation est relativement court, permettant au système d'atteindre rapidement la valeur cible de 1 imposée par l'entrée échelon. Par ailleurs, nous constatons absences d'instabilité ou de bruit sur l'allure de la courbe de sortie pour atteindre la valeur souhaitée. Ainsi, cela met en évidence l'efficacité de la commande B à assurer une convergence rapide vers la consigne tout en maintenant la stabilité du système.

5. Commande adaptative B

Dans le cadre de ce projet, nous avons étudié et appliqué la commande adaptative ainsi que la commande B séparément à notre système. Cette section va donc se concentrer sur l'étude de l'association de ces deux commandes pour aboutir à la commande B adaptative. L'objectif de cette association est de combiner les avantages de chacune des commandes afin qu'elles se complètent dans leurs tâches respectives. Pour appliquer cette approche, nous avons conçu et réalisé le schéma bloc du système intégrant cette nouvelle commande sur le Simulink. Ce schéma nous a permis de modéliser et d'analyser le comportement du système sous l'effet de la commande B adaptative (voir figure 32).

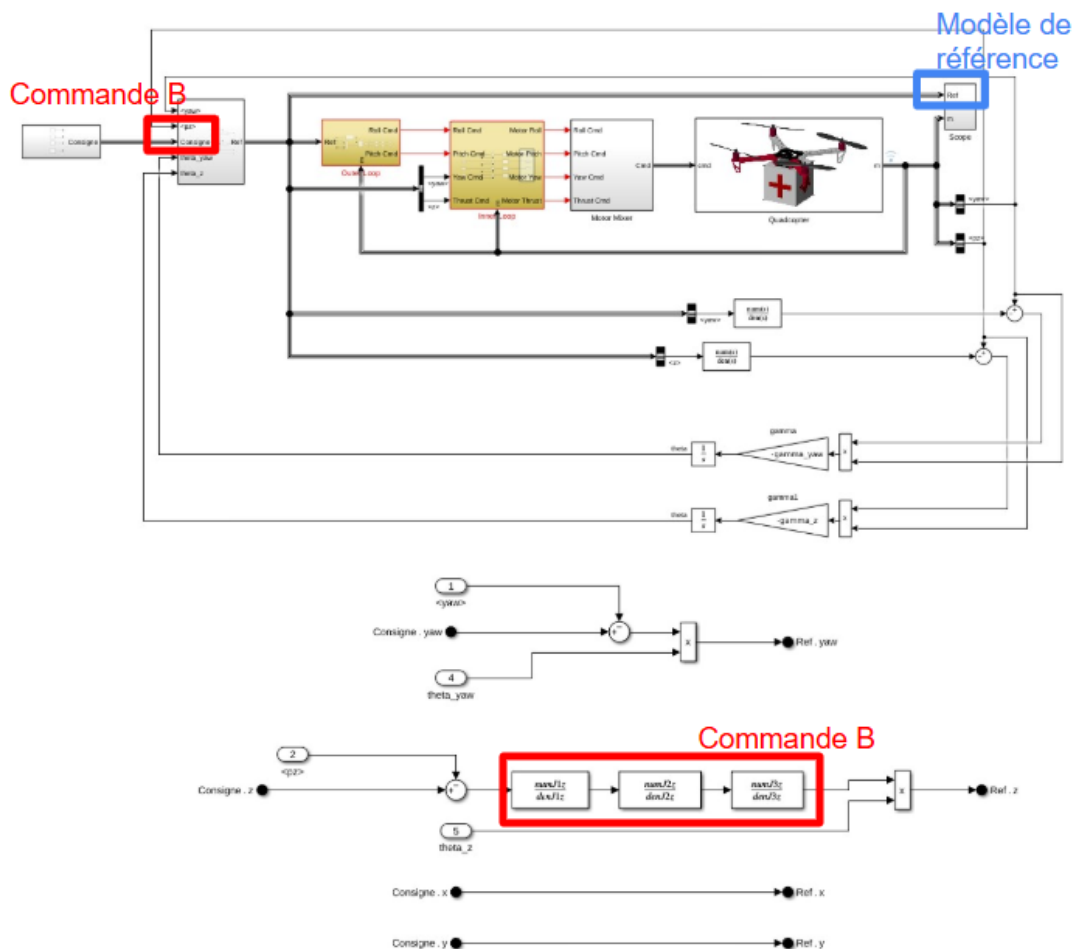


Figure 33 : Schéma bloc du système du drone avec la commande B adaptative sur Simulink

Tout d'abord, nous pouvons observer sur notre schéma les blocs de la boucle extérieure (outer loop), de la boucle intérieure (inner loop) et du moteur mixer en amont du système, qui constituent l'architecture de contrôle du drone. Ensuite, nous retrouvons le bloc représentant le modèle de notre quadricoptère. Puis, nous avons notre première boucle de retour avec la présence du bloc symbolisant système de référence encadré en bleu. Cette partie permet à notre système de comparer sa sortie à un modèle idéal sans perturbation que nous souhaitons atteindre. Dans la deuxième boucle de retour, se trouve le bloc d'apprentissage, qui permet à notre modèle de s'adapter en suivant la règle MIT que nous avons choisie. Enfin, à la suite du bloc d'apprentissage au niveau de la consigne, nous avons ajouté notre compensateur B pour l'axe z encadré en rouge. Il va corriger l'entrée du système en fonction des données fournies par le bloc d'apprentissage, afin d'atteindre la sortie souhaitée le plus rapidement possible, tout en minimisant les perturbations et l'instabilité. Ainsi, nous avons intégré notre commande B adaptative dans le modèle du drone pour voir les résultats obtenus (voir figure 33).

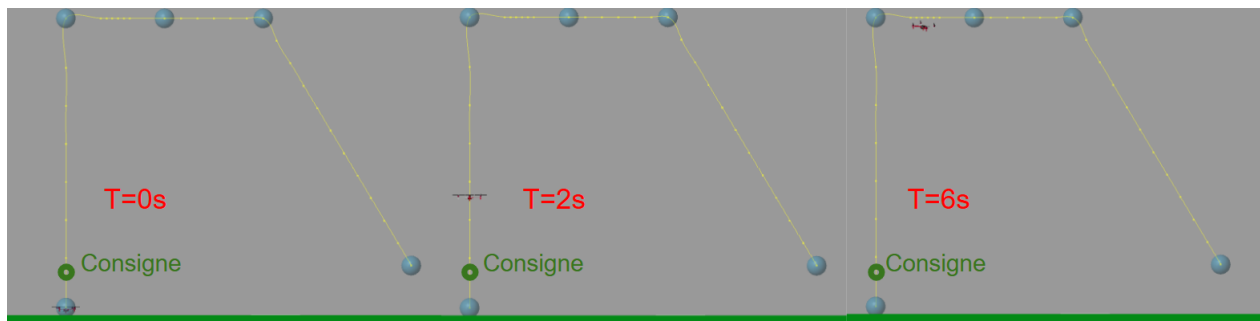


Figure 34 : Simulation du drone avec la commande B adaptative sur Matlab

En observant la simulation du système sur Matlab, nous constatons que le drone tente de se stabiliser rapidement à la position de consigne souhaitée. Cependant, au bout de 6 secondes d'intervalle de simulation, il décolle et dérive vers la droite sur l'axe x. Cette anomalie est probablement due à la complexité du système qui se trouve être un système multilinéaire. De plus, le modèle du drone est un bloc contenant des fonctions mathématiques complexes auxquelles nous n'avons pas forcément accès. Cela rend donc difficile l'adaptation de nos commandes à ce modèle. Par ailleurs, la commande B que nous avons implémentée repose sur les travaux de M. Bensoussan. Cependant, les simplifications proposées dans son article de recherche ne semblent pas adaptées à ce cas précis. Il serait donc nécessaire de recalibrer les paramètres de $J(s)$ de la commande B afin de permettre au drone de se stabiliser correctement sans incohérences. L'objectif de cette partie étant de réussir à implanter correctement la commande B adaptative avec des résultats probants, elle reste inachevée sur le délai imparti. Une prochaine étape pour compléter cette partie du projet serait de réussir à obtenir les résultats souhaités avec notre commande B adaptative sur le modèle Matlab. Ensuite, nous pourrions appliquer cette approche au drone réel et comparer les résultats obtenus avec ceux des autres méthodes de commande.

6. Conclusion

Pour conclure, nous avons pu approfondi au travers de ce projet notre compréhension des méthodologies de commande, en particulier sur le travail de M.Bensoussan sur la commande B. Ce travail nous a également permis de développer nos compétences dans divers domaines tels que Matlab, GeoGebra, la gestion de projet, la conception et le fonctionnement d'un drone. Bien que le drone ait été assemblé, il n'a pas pu être mis en marche en raison d'un retard dans la réception de composants clé. Par ailleurs, nous avons pu élaborer un modèle complet du système intégrant la commande B, MRAC, et B adaptative, mais le temps nous a manqué pour affiner les paramètres de $J(s)$ de la commande B. L'affinement de ces paramètres sont nécessaires pour obtenir des résultats optimaux pour notre système.

Ce projet a été bénéfique pour notre apprentissage, mais certains éléments ont fait défaut pour mener à bien tous les objectifs fixés. La contrainte de temps nous a notamment limité notre capacité à finaliser certains aspects cruciaux. Une suite logique de ce travail consisterait à ajuster la commande B adaptative sur Matlab pour obtenir des résultats conformes aux attentes. Ensuite, il serait pertinent de finaliser l'assemblage du drone afin de le mettre en marche et de lui appliquer les différentes commandes développées au cours du projet. Enfin, nous obtenir des résultats dans des conditions réelles et juger de l'intérêt de la commande B adaptative.

7. Bibliographie

Brossard, J. (s.d.). *A new fast compensator design applied to a quadcopter*. [Article].

Lin, H.-C. (s.d.). *COMP514* [Cours]. McGill University.

MathWorks. (s.d.). *Cours MATLAB sur la commande classique d'un drone*. MathWorks.

<https://www.mathworks.com/videos/series/drone-simulation-and-control.html>

MathWorks. (s.d.). *Exemple MATLAB drone MRAC*. MathWorks.

<https://www.mathworks.com/help/slcontrol/ug/quadrotor-control-using-model-reference-adaptive-control.html>

MathWorks. (s.d.). *Model Reference Adaptive Control (MRAC) avec MATLAB*. MathWorks.

<https://www.mathworks.com/help/slcontrol/ug/model-reference-adaptive-control.html>

MathWorks. (s.d.). *Simple Adaptive Control Example*. MATLAB Central.

<https://www.mathworks.com/matlabcentral/fileexchange/44416-simple-adaptive-control-example>

ResearchGate. (s.d.). *Figure for the MIT rule of the Model Reference Adaptive Control*.

https://www.researchgate.net/figure/Model-reference-adaptive-control-based-on-MIT-rule_fig1_345670478

ROS Wiki. (s.d.). *ROS Rotors Simulator*. ROS Wiki. https://wiki.ros.org/rotors_simulator

YouTube. (s.d.). *Simulation du quadricoptère* [Vidéo]. <https://youtu.be/x9dKYkGfaY4>

Ghodbane, A., Bensoussan, D., & Hammami, M. (2023). Improved simultaneous performance in the time domain and in the frequency domain.

Bensoussan, M. (s.d.). *SYS802 : Méthodes Avancées de Commandes* [Cours]. École de technologie supérieure.

8. Annexes

8.1. Sous modèle du code simulink

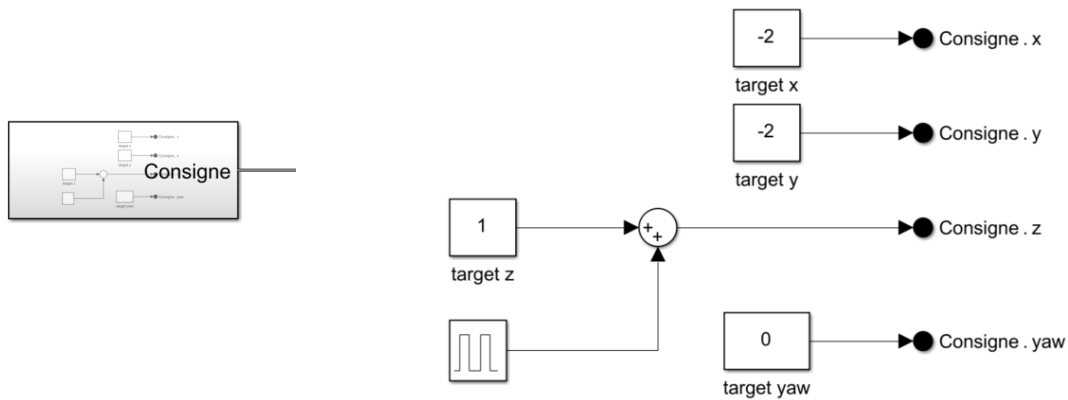


Figure 35 : Sous-modèle Consigne

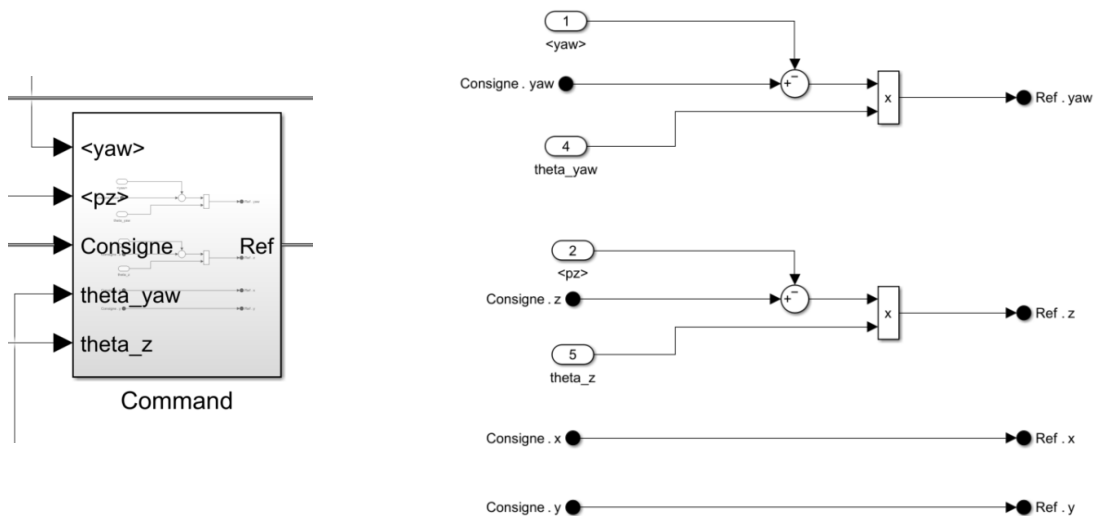


Figure 36 : Sous-modèle Command sans la commande B

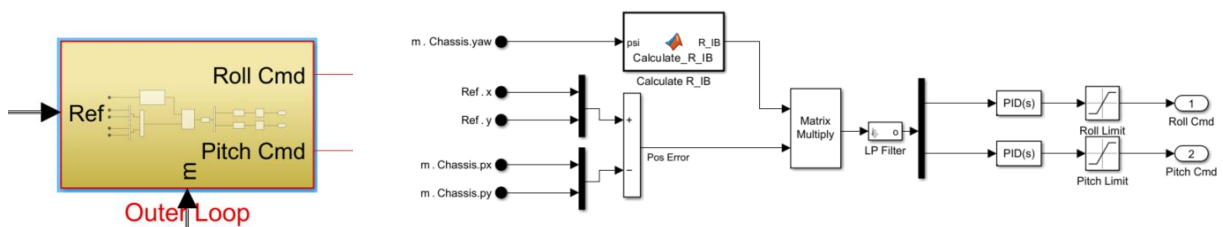


Figure 37 : Sous-modèle Outer Loop

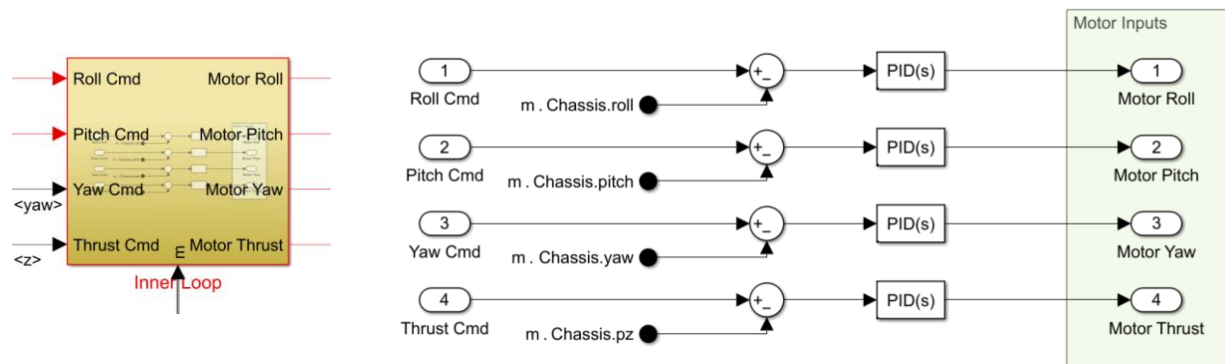


Figure 38 : Sous-modèle Inner Loop

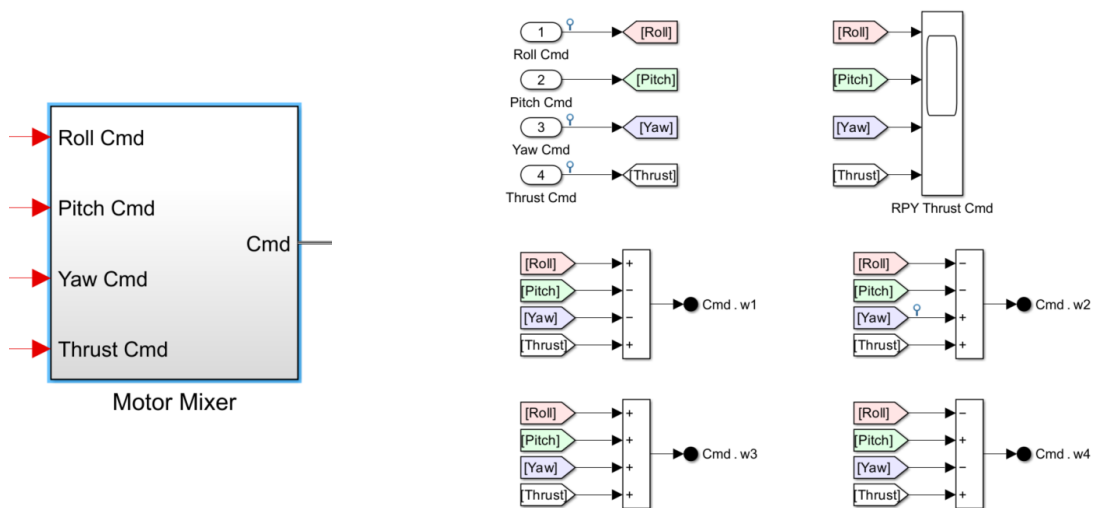


Figure 39 : Sous-modèle Motor Mixer

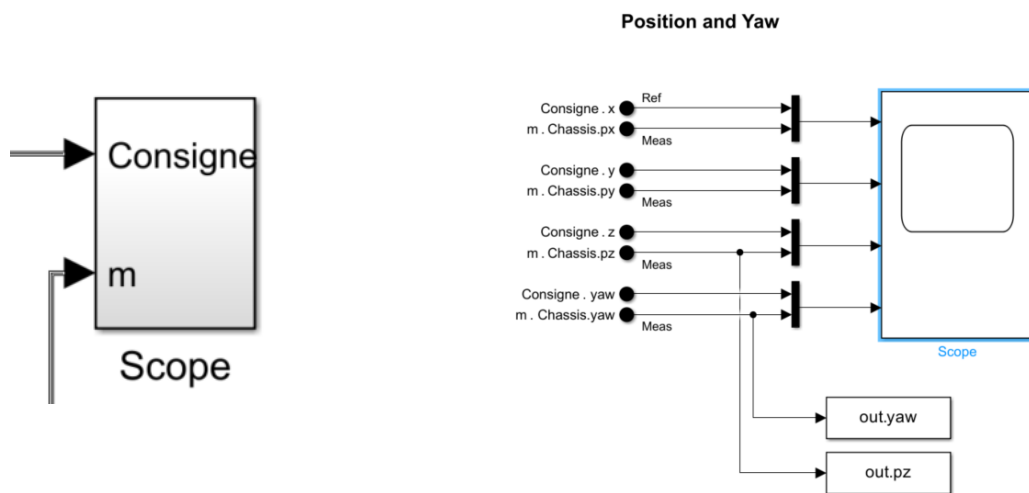


Figure 40 : Sous-modèle Scope

8.2. SystemIdentification.m

```
close all;

% Extraire les données pour yaw et pz
yaw_time = out.yaw.Time;
yaw_values = out.yaw.Data;

pz_time = out.pz.Time;
pz_values = out.pz.Data;

index_yaw = 422; %% Starting index for yaw
index_pz = 445; %% Starting index for z

filtered_yaw_time = yaw_time(index_yaw:end);
filtered_yaw_values = yaw_values(index_yaw:end);

filtered_yaw_time = filtered_yaw_time - filtered_yaw_time(1);
filtered_yaw_values = filtered_yaw_values - filtered_yaw_values(1);

filtered_pz_time = pz_time(index_pz:end);
filtered_pz_values = pz_values(index_pz:end);

filtered_pz_time = filtered_pz_time - filtered_pz_time(1);
filtered_pz_values = filtered_pz_values - filtered_pz_values(1);

% Tracer les réponses à un échelon
figure;

% Réponse pour Yaw
subplot(2, 1, 1);
plot(filtered_yaw_time, filtered_yaw_values, 'b', 'LineWidth', 1.2);
title('Réponse à un échelon - Yaw');
xlabel('Temps (s)');
ylabel('Amplitude');
grid on;

% Réponse pour Pz
subplot(2, 1, 2);
plot(filtered_pz_time, filtered_pz_values, 'r', 'LineWidth', 1.2);
title('Réponse à un échelon - Pz');
xlabel('Temps (s)');
ylabel('Amplitude');
grid on;

sgtitle('Réponses à un échelon');

% Périodes d'échantillonnage
Ts_yaw = filtered_yaw_time(2) - filtered_yaw_time(1);
Ts_pz = filtered_pz_time(2) - filtered_pz_time(1);

% Créer les données iddata
data_yaw = iddata(filtered_yaw_values, ones(size(filtered_yaw_values)), Ts_yaw);
```

```

data_pz = iddata(filtered_pz_values, ones(size(filtered_pz_values)), Ts_pz);

% Identifier un modèle d'ordre 2 pour Yaw
[sys_yaw, yaw_fit_info] = tfest(data_yaw, 2, 0) % Modèle d'ordre 2, sans zéro

% Identifier un modèle d'ordre 2 pour Pz
[sys_pz, pz_fit_info] = tfest(data_pz, 2, 0) % Modèle d'ordre 2, sans zéro

figure;
subplot(2, 1, 1);
step(sys_yaw, 9);
title('Réponse à un échelon - Yaw');
xlabel('Temps (s)');
ylabel('Amplitude');
grid on;

% Réponse pour Pz
subplot(2, 1, 2);
step(sys_pz, 9);
title('Réponse à un échelon - Pz');
xlabel('Temps (s)');
ylabel('Amplitude');
grid on;

```

8.3. Annexe 2

```

% Script pour définir les variables inconnues de J1(s) et J3(s)
s=tf('s');
% Définir les paramètres de J1(s) pour x et y
k1i = 53; % Gain k1i (valeur à ajuster)
omega1i = 0.07; % Fréquence omega1i (valeur à ajuster)

% Définir les paramètres de J3(s)
omega2i = 34; % Fréquence omega2i (valeur à ajuster)
ki = 4; % Exposant ki (valeur à ajuster)

% Définir les paramètres de J2(s)
z = 0.02;
p = 0.08;

% Définir les paramètres de J1(s) pour z
k1i_z = 38; % Gain k1i (valeur à ajuster)
omega1i_z = 0.1; % Fréquence omega1i (valeur à ajuster)

% Définir les paramètres de J3(s) pour z
omega2i_z = 18; % Fréquence omega2i (valeur à ajuster)
ki_z = 2; % Exposant ki (valeur à ajuster)

% Définir les paramètres de J2(s) pour z

```

```

z_z = 0.1;
p_z = 0.015;

% Définir les paramètres de J1(s) pour le lacet
k1i_psi = 18;           % Gain k1i (valeur à ajuster)
omega1i_psi = 0.9;      % Fréquence omega1i (valeur à ajuster)

% Définir les paramètres de J3(s) pour le lacet
omega2i_psi = 77;       % Fréquence omega2i (valeur à ajuster)
ki_psi = 2;             % Exposant ki (valeur à ajuster)

% Définir les paramètres de J2(s) pour le lacet
z_psi = 0.1;
p_psi = 0.01;

% Formule du J1(s)_xy
J1xy=k1i*omega1i/(s+omega1i);

% Formule de J3(s)_xy
J2xy=(omega2i/(s+omega2i))^ki;

% Formule de J2(s)_xy
J3xy=(s+z)/(s+p);

% Formule du J1(s)_z
J1z=k1i_z*omega1i_z/(s+omega1i_z);

% Formule de J3(s)_z
J2z=(omega2i_z/(s+omega2i_z))^ki_z;

% Formule de J2(s)_z
J3z=(s+z_z)/(s+p_z);

% Formule du J1(s)_psi axe lacet
J1psi=k1i_psi*omega1i_psi/(s+omega1i);

% Formule de J3(s)_psi axe lacet
J2psi=(omega2i_psi/(s+omega2i_psi))^ki_psi;

% Formule de J2(s)_psi axe lacet
J3psi=(s+z_psi)/(s+p_psi);

```

8.4. Annexe 3

```

%%Récupération du numérateur et dénominateur de la fonction de transfert J1 pour
l'axe x/y
[numJ1xy,denJ1xy]=tfdata(J1xy,"v");
%%Récupération du numérateur et dénominateur de la fonction de transfert J2 pour
l'axe x/y
[numJ2xy,denJ2xy]=tfdata(J2xy,"v");
%%Récupération du numérateur et dénominateur de la fonction de transfert J3 pour
l'axe x/y

```



```

[numJ3xy,denJ3xy]=tfdata(J3xy,"v");

%%Récupération du numérateur et dénominateur de la fonction de transfert J1 pour
l'axe z
[numJ1z,denJ1z]=tfdata(J1z,"v");
%%Récupération du numérateur et dénominateur de la fonction de transfert J2 pour
l'axe z
[numJ2z,denJ2z]=tfdata(J2z,"v");
%%Récupération du numérateur et dénominateur de la fonction de transfert J3 pour
l'axe z
[numJ3z,denJ3z]=tfdata(J3z,"v");

%%Récupération du numérateur et dénominateur de la fonction de transfert J1 pour
l'axe des lacets
[numJ1psi,denJ1psi]=tfdata(J1psi,"v");
%%Récupération du numérateur et dénominateur de la fonction de transfert J2 pour
l'axe des lacets
[numJ2psi,denJ2psi]=tfdata(J2psi,"v");
%%Récupération du numérateur et dénominateur de la fonction de transfert J3 pour
l'axe des lacets
[numJ3psi,denJ3psi]=tfdata(J3psi,"v");

```